

北京理工大学

BEIJING INSTITUTE OF TECHNOLOGY



计算机组成与体系结构课 程 设 计

个人实验报告

学 院	计算机学院
专 业	计算机科学与技术（拔尖班）
指导老师	罗兆经
姓 名	周泽顺
学 号	1120230501
联系方式	1120230501@bit.edu.cn

二〇二六年 六 月

目 录

第一章 项目简述

第二章 设计环境

第三章 设计原理及内容

第四章 设计与实现

第五章 测试

第六章 心得体会及总结

第一章 项目简述

本项目设计并实现了一种单周期 RISC-V 处理器，支持 RV32I 整数指令集中的 ADD、SUB、ORI、LW、SW、BEQ 等指令。处理器采用模块化设计，包含 PC、指令存储器、寄存器堆、立即数生成器、ALU、数据存储器 and 控制器等核心模块，数据通路在一个时钟周期内完成取指、译码、执行、访存和写回。

测试程序将 5 个整数的冒泡排序的汇编代码输入 RARS 仿真器中调试并导出机器码。将机器码加载至所设计 CPU 的指令存储器中进行仿真，仿真结果与 RARS 运行结果完全一致。通过本项目，能够掌握 RISC-V 指令格式与编码，掌握 RISC-V 汇编转换成机器代码的方法，掌握单周期 CPU 的数据通路与控制器设计方法。

第二章 设计环境

操作系统	Windows 11
编程语言	Verilog HDL
EDA 工具	Vivado 2019.2
汇编语言	RISC-V
汇编程序编辑器	RARS 1.6; VS Code

第三章 设计原理及内容

CPU 整体架构

本单周期 CPU 采用模块化设计，通过 7 个核心模块的协作和通信完成计算和控制任务。数据通路包含 PC、指令存储器、寄存器堆、ALU、数据存储器和控制单元。每条指令在一个时钟周期内依次完成取指、译码、执行、访存和写回五阶段。控制器采用组合逻辑硬连线，根据 opcode、funct3、funct7 生成所有控制信号，确保指令正确执行。

模块	功能描述
PC	存储当前指令地址，每个时钟上升沿更新为下一条指令的地址（顺序执行或跳转目标）。复位初值为 0x00400000（RISC-V 程序入口）。
InstructionMemory	只读存储器，存储用户程序机器码。通过地址 <code>addr</code> 读取 32 位指令。采用 <code>\$readmemh</code> 从外部 .hex 文件加载指令，支持灵活更换测试程序。地址映射： <code>mem[(addr - 0x00400000) >> 2]</code> 。
RegisterFile	32 个 32 位通用寄存器（x0~x31），x0 硬连线为 0。支持两个异步读端口和一个同步写端口。写操作在时钟上升沿有效，且 x0 不可写入。
ImmediateGenerator	根据 <code>imm_type</code> 信号，从 32 位指令码中提取并符号扩展立即数。支持 I 型、S 型、B 型、U 型、J 型五种格式。
ALU	算术逻辑单元，支持加法（0000）、减法（0001）、按位或（0010）、按位与（0011）、异或（0100）、小于置位（0101）、逻辑左移（0110）。输出运算结果和零标志 <code>zero</code> 。
DataMemory	可读写存储器，容量 1024 字（4KB）。同步写、异步读。基地址 0x10010000（与 RARS 默认数据段对齐），地址超出范围时读返回 0，写被忽略。通过 <code>\$readmemh</code> 初始化数据段。
ControlUnit	控制总线，根据指令的 <code>opcode</code> 、 <code>funct3</code> 、 <code>funct7</code> 字段译码，生成所有控制信号。

表 3.1-1 单周期 CPU 模块说明

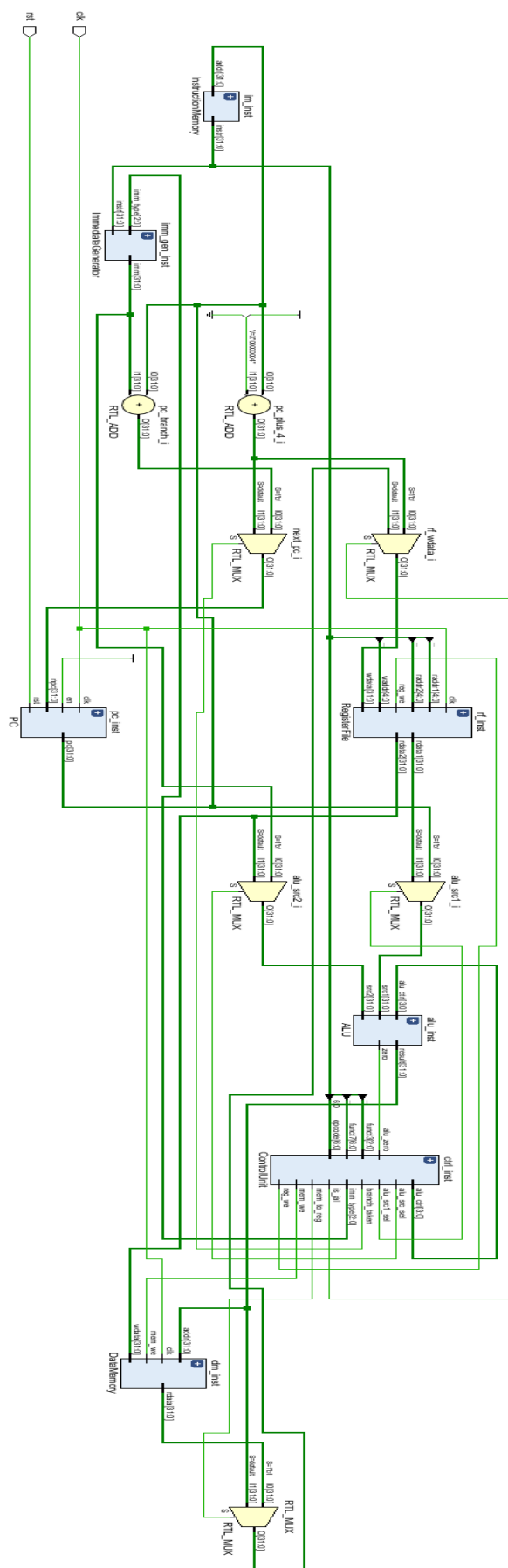


图 3.1-2 单周期 CPU 整体架构图

序号	指令	opcode	funct3	funct7	功能
1	add	0110011	000	0000000	加法运算: $rd = rs1 + rs2$
2	sub	0110011	000	0100000	减法运算: $rd = rs1 - rs2$
3	slt	0110011	010	0000000	小于置位: $rd = (rs1 < rs2) ? 1 : 0$
4	addi	0010011	000	-	立即数加法: $rd = rs1 + imm$
5	ori	0010011	110	-	立即数按位或: $rd = rs1 imm$
6	slli	0010011	001	-	逻辑左移: $rd = rs1 \ll imm[4:0]$
7	lw	0000011	010	-	加载字: $rd = Mem[rs1 + imm]$
8	sw	0100011	010	-	存储字: $Mem[rs1 + imm] = rs2$
9	beq	1100011	000	-	相等跳转: if $(rs1 == rs2)$ $PC = PC + imm$
10	blt	1100011	100	-	小于跳转: if $(rs1 < rs2)$ $PC = PC + imm$
11	bge	1100011	101	-	大于等于跳转: if $(rs1 \geq rs2)$ $PC = PC + imm$
12	auipc	0010111	-	-	PC 加高位立即数: $rd = PC + (imm \ll 12)$
13	jal	1101111	-	-	跳转并链接: $rd = PC + 4$; $PC = PC + imm$

表 3.1-3 单周期 CPU 实现的 13 条指令

3.1.1 数据通路

本单周期 CPU 的数据通路在一个时钟周期内完成一条指令的全部操作。整体工作流程如下:

1. 取指 (Fetch): 程序计数器 PC 寄存器的当前值 `curr_pc` 作为地址, 送入指令存储器 `InstructionMemory`。指令存储器根据相对地址 (PC 基址为 `0x00400000`) 读出对应的 32 位机器码 `instr`。同时, PC 的下一时钟周期值 `next_pc` 由 PC 计算逻辑生成: 默认顺序执行时为 `curr_pc + 4`; 若发生分支或跳转, 则采用 `curr_pc + imm`。
2. 译码 (Decode): 指令 `instr` 的 `opcode`、`funct3`、`funct7` 字段送入控制总线 `ControlUnit`, 经组合逻辑立即产生所有控制信号。同时, 指令中的 `rs1` 和 `rs2` 字段作为读地址, 访问寄存器堆 `RegisterFile`。寄存器堆异步读出两个源操作数 `rdata1` 和 `rdata2`。指令中的立即数相关位段与 `imm_type` 信号一并送入立即数生成器 `ImmediateGenerator`, 得到符号扩展后的 32 位立即数 `imm_out`。
3. 执行 (Execute): ALU 的两个源操作数分别通过多路选择器选择: 第一操作数 `alu_src1` 由 `alu_src1_sel` 选择: 0 为 `rdata1` (来自寄存器), 1 为 `curr_pc` (用于 `auipc`)。第二操作数 `alu_src2` 由 `alu_src2_sel` 选择: 0 为 `rdata2`,

1 为 `imm_out`。ALU 根据 `alu_ctrl` 信号执行相应运算（如加法、减法、按位或、小于置位、左移等），输出 `alu_result` 和零标志 `alu_zero`。

4. 访存（Memory）：ALU 的计算结果 `alu_result` 作为地址，连同 `mem_we`、写数据 `rdata2` 送入数据存储器 `DataMemory`。若 `mem_we` 有效（`sw` 指令），则数据被写入对应地址；否则读出 `mem_rdata`。数据存储器的基地址为 **0x10010000**，与 RARS 默认数据段对齐。
5. 写回（Write Back）：写回数据 `wb_data` 由 `mem_to_reg` 信号选择：0 时取自 `alu_result`（R 型、I 型、`auipc`），1 时取自 `mem_rdata`（`lw` 指令）。对于 `jal` 指令，写回的是 `PC+4`（通过顶层额外选择器实现）。若 `reg_we` 有效且写地址 `rd` 不为 0，寄存器堆在时钟上升沿将 `wb_data` 写入 `rd`。

分支跳转：`branch_taken` 信号由控制单元根据指令类型和 ALU 输出综合产生（如 `beq` 要求 `alu_zero=1`；`blt` 要求 `slt` 结果为 1；`bge` 要求 `slt` 结果为 0；`jal` 无条件跳转）。当 `branch_taken` 有效时，`next_pc` 采用分支地址，否则顺序执行。

数据通路的各个信号与模块的协作关系于图 3.1-2 中展示。

3.1.2 控制逻辑

控制总线产生的核心控制信号包括：

- `reg_we`：寄存器写使能信号，控制是否更新寄存器堆。
- `mem_we`：存储器写使能信号，控制是否将数据存入内存（用于 `sw`）。
- `mem_to_reg`：写回数据来源选择信号，区分写入寄存器的是 ALU 结果还是内存数据。
- `alu_ctrl [3:0]`：ALU 操作码，指示执行哪种具体的算术逻辑运算。
- `alu_src_sel`：ALU 第二操作数选择，在寄存器 `rs2` 输出和立即数 `imm` 之间切换。
- `alu_src1_sel`：ALU 第一操作数选择，在寄存器 `rs1` 输出和当前 `PC` 之间切换（用于 `auipc`）。
- `branch_taken`：跳转判定控制信号，结合 ALU 的 `zero` 信号决定是否执行分支或跳转。
- `imm_type [2:0]`：立即数类型控制，指示 `ImmGen` 按照哪种指令格式解析立即数。
- `is_jal`：`jal` 指令专用信号，用于确保链接地址 `PC+4` 正确写入寄存器。

第四章 设计与实现

4.1 控制总线

控制总线负责将 32 位机器码转换为处理器内部的控制信号，核心逻辑为指令译码和控制信号生成两部分。对指令类型的定义存放在 `macro.vh` 头文件中。

```
module ControlUnit (
    input  [6:0] opcode,
    input  [2:0] funct3,
    input  [6:0] funct7,
    input      alu_zero,
    output     reg_we,
    output     mem_we,
    output     mem_to_reg,
    output [3:0] alu_ctrl,
    output     alu_src_sel,
    output     alu_src1_sel,
    output     branch_taken,
    output [2:0] imm_type,
    output     is_jal
);
    // decode
    wire is_add  = (opcode == `OP_R_TYPE) && (funct3 == 3'b000) && (funct7 ==
7'b0000000);
    wire is_sub  = (opcode == `OP_R_TYPE) && (funct3 == 3'b000) && (funct7 ==
7'b0100000);
    wire is_addi = (opcode == `OP_I_TYPE) && (funct3 == 3'b000);
    wire is_ori  = (opcode == `OP_I_TYPE) && (funct3 == 3'b110);
    wire is_slli = (opcode == `OP_I_TYPE) && (funct3 == 3'b001);
    wire is_lw   = (opcode == `OP_LOAD)  && (funct3 == 3'b010);
    wire is_sw   = (opcode == `OP_STORE) && (funct3 == 3'b010);
    wire is_beq  = (opcode == `OP_BRANCH) && (funct3 == 3'b000);
    wire is_blt  = (opcode == `OP_BRANCH) && (funct3 == 3'b100);
    wire is_bge  = (opcode == `OP_BRANCH) && (funct3 == 3'b101);
    wire is_slt  = (opcode == `OP_R_TYPE) && (funct3 == 3'b010) && (funct7 ==
7'b0000000);
    assign is_jal = (opcode == `OP_JAL);
    wire is_auipc = (opcode == `OP_AUIPC);
    // control signals
    assign reg_we      = is_add | is_sub | is_addi | is_ori | is_lw | is_slt |
is_slli | is_auipc | is_jal;
    assign mem_we      = is_sw;
```

```

assign mem_to_reg = is_lw;
assign alu_src_sel = is_addi | is_ori | is_lw | is_sw | is_auipc | is_slli;
assign alu_src1_sel = is_auipc;
assign branch_taken = (is_beq && alu_zero) || (is_blt && !alu_zero) || (is_bge
&& alu_zero) || is_jal;
assign imm_type = (is_sw) ? 3'b001 :
                  (is_beq || is_blt || is_bge) ? 3'b010 :
                  (is_auipc) ? 3'b011 :
                  (is_jal) ? 3'b100 : 3'b000;

// ALU control
assign alu_ctrl = (is_add | is_addi | is_lw | is_sw | is_auipc) ? 4'b0000 :
                  (is_sub | is_beq) ? 4'b0001 :
                  is_ori ? 4'b0010 :
                  (is_slt | is_blt | is_bge) ? 4'b0101 :
                  is_slli ? 4'b0110 :
                  4'b1111;

endmodule

```

图 4.1-1 ControlUnit 代码

4.2 寄存器堆读写

寄存器堆实现 32 个 32 位寄存器，支持异步读、同步写，x0 硬连线为 0。该模块展示了数据通路如何工作并完成读写工作。

```

module RegisterFile (
    input      clk,
    input      reg_we,          // 写使能
    input [4:0] raddr1,         // 读端口 1 地址
    input [4:0] raddr2,         // 读端口 2 地址
    input [4:0] waddr,          // 写地址
    input [31:0] wdata,          // 写数据
    output [31:0] rdata1,        // 读数据 1
    output [31:0] rdata2        // 读数据 2
);
    // 写操作（同步，x0 不可写）
    always @(posedge clk) begin
        if (reg_we && (waddr != 0))
            regs[waddr] <= wdata;
    end

    // 读操作（组合逻辑，x0 读 0）
    assign rdata1 = (raddr1 == 0) ? 32'b0 : regs[raddr1];
    assign rdata2 = (raddr2 == 0) ? 32'b0 : regs[raddr2];
endmodule

```

图 4.1-2 寄存器堆读写代码

4.3 单周期 CPU 顶层模块

顶层模块实例化所有子模块，并通过多路选择器和 PC 更新逻辑连接数据通路。

```
module SingleCycleCPU(  
    input clk,  
    input rst  
);  
    // Wires for components  
    wire [31:0] curr_pc;  
    wire [31:0] next_pc;  
    wire [31:0] instruction;  
  
    wire [4:0] rs1 = instruction[19:15];  
    wire [4:0] rs2 = instruction[24:20];  
    wire [4:0] rd = instruction[11:7];  
    wire [6:0] opcode = instruction[6:0];  
    wire [2:0] funct3 = instruction[14:12];  
    wire [6:0] funct7 = instruction[31:25];  
  
    wire [31:0] rdata1, rdata2;  
    wire [31:0] alu_result;  
    wire [31:0] mem_rdata;  
    wire alu_zero;  
  
    // Control Signals  
    wire reg_we;  
    wire mem_we;  
    wire mem_to_reg;  
    wire [3:0] alu_ctrl;  
    wire branch_taken;  
  
    // --- PC ---  
    wire [31:0] pc_plus_4 = curr_pc + 4;  
    wire [31:0] pc_branch = curr_pc + imm_out;  
    assign next_pc = branch_taken ? pc_branch : pc_plus_4;  
    // ...  
    // --- Register File ---  
    wire is_jal;  
    wire [31:0] rf_wdata = is_jal ? pc_plus_4 : (mem_to_reg ? mem_rdata :  
alu_result);
```

```

RegisterFile rf_inst (
    .clk(clk),
    .reg_we(reg_we),
    .raddr1(rs1),
    .raddr2(rs2),
    .waddr(rd),
    .wdata(rf_wdata),
    .rdata1(rdata1),
    .rdata2(rdata2)
);
// --- ALU ---
wire [31:0] alu_src1;
wire [31:0] alu_src2;
wire alu_src_sel; // 0: rdata2, 1: immediate
wire alu_src1_sel; // 0: rdata1, 1: pc

assign alu_src1 = alu_src1_sel ? curr_pc : rdata1;
assign alu_src2 = alu_src_sel ? imm_out : rdata2;

ALU alu_inst (
    .src1(alu_src1),
    .src2(alu_src2),
    .alu_ctrl(alu_ctrl),
    .result(alu_result),
    .zero(alu_zero)
);
// ...
endmodule

```

图 4.1-3 单周期 CPU 顶层模块代码

第五章 测试

5.1 单元测试

在将 CPU 的各个子模块组装之前，实验先对各模块的功能展开简单的单元测试。各单元测试的内容如下表所示。所有模块均通过了单元测试，证明模块运行的正确性。

模块	测试文件	测试内容	测试结果
PC + 指令存储器	tb_PC_IM.v	PC 从 0x00400000 开始，每周+4，指令存储器正确读出指令	PC 递增正常，指令输出正确
寄存器堆	tb_RegisterFile.v	写入 x1=0x12345678，再读出；尝试写入 x0 无效	写入后读出值正确，x0 始终为 0
ALU	tb_ALU.v	测试 ADD、SUB、OR 运算（5+3=8, 5-3=2, 5 3=7）	计算结果与预期一致，zero 标志正确
立即数生成器	tb_ImmGen.v	测试 I/S/B 型立即数提取和符号扩展	立即数值与 RISC-V 规范一致

表 5.1-1 单元测试用例

<pre># run 1500ns time=0, pc=xxxxxxxx, instr=xxxxxxxx time=5000, pc=00400000, instr=0fc10417 time=15000, pc=00400004, instr=00040413 time=25000, pc=00400008, instr=0fc10497 time=35000, pc=0040000c, instr=00c4a483 time=45000, pc=00400010, instr=fff48913 time=55000, pc=00400014, instr=02094e63 time=65000, pc=00400018, instr=00000293 time=75000, pc=0040001c, instr=0322d663 time=85000, pc=00400020, instr=00229313 time=95000, pc=00400024, instr=006403b3 time=105000, pc=00400028, instr=0003ae03 time=115000, pc=0040002c, instr=0043ae83 time=125000, pc=00400030, instr=01ceaf33 time=135000, pc=00400034, instr=000f0663 time=145000, pc=00400038, instr=01d3a023 time=155000, pc=0040003c, instr=01c3a223</pre>	<pre># run 1500ns ADD: 5 + 3 = 8 SUB: 5 - 3 = 2 OR: 5 3 = 7 # run 1500ns time=0, reg_we=1, waddr= 1, wdata=12345678, rdata1=xxxxxxxx time=10000, reg_we=0, waddr= 1, wdata=12345678, rdata1=12345678 time=20000, reg_we=1, waddr= 0, wdata=ffffffff, rdata1=12345678 time=30000, reg_we=0, waddr= 0, wdata=ffffffff, rdata1=00000000 # run 1500ns I imm = 00000080 S imm = 00000080 B imm = 00000008</pre>
--	--

图 5.1-2 单元测试结果（左：PC+IM；右上：ALU；右中：寄存器；右下：立即数生成器）

5.2 系统测试

系统测试采用 tb_CPU.v 作为顶层仿真测试平台。使用 RISC-V 汇编编写 5 个整数排序程序，导出 RARS 仿真的机器码与数据段内容，经过脚本得到能为所设计 CPU 读取的代码与数据文件。随后在 Vivado 中再次进行仿真，比较结果。

5.2.1 测试程序

使用 RISC-V 汇编编写冒泡排序程序。当前程序的输入为(9,3,7,1,5)，最终预计结果为数组内存中共五个 2 字节的位置上数字被从小到大排序。

```
# sort.asm - 冒泡排序，使用 SLT 实现小于比较
.data
arr:  .word  9, 3, 7, 1, 5
n:    .word  5
.text
.globl main
main:
    la    s0, arr
    lw    s1, n          # s1 = 5
    addi   s2, s1, -1     # outer = 4
outer:
    blt    s2, zero, done # if outer < 0, exit
    li     t0, 0          # j = 0
inner:
    bge     t0, s2, outer_next # if j >= outer, break
    sllli   t1, t0, 2
    add     t2, s0, t1
    lw      t3, 0(t2)      # arr[j]
    lw      t4, 4(t2)      # arr[j+1]
    slt     t5, t4, t3     # t5 = (arr[j+1] < arr[j]) ? 1 : 0
    beq     t5, zero, no_swap # if arr[j+1] >= arr[j], no swap
    sw      t4, 0(t2)      # swap
    sw      t3, 4(t2)
no_swap:
    addi    t0, t0, 1
    j       inner
outer_next:
    addi    s2, s2, -1
    j       outer
done:
    # 输出排序结果（可选）
    li     t0, 0
```

图 5.2-1 排序程序汇编代码

从 RARS 仿真器导出指令的机器码与数据布局。为了使 CPU 能够正确读入，使用脚本 `translate_inst.py` 和 `translate_mem.py` 提取清洗后的指令机器码和所有使用过的内存数据。文件读写路径在 `Lab2.file` 中。

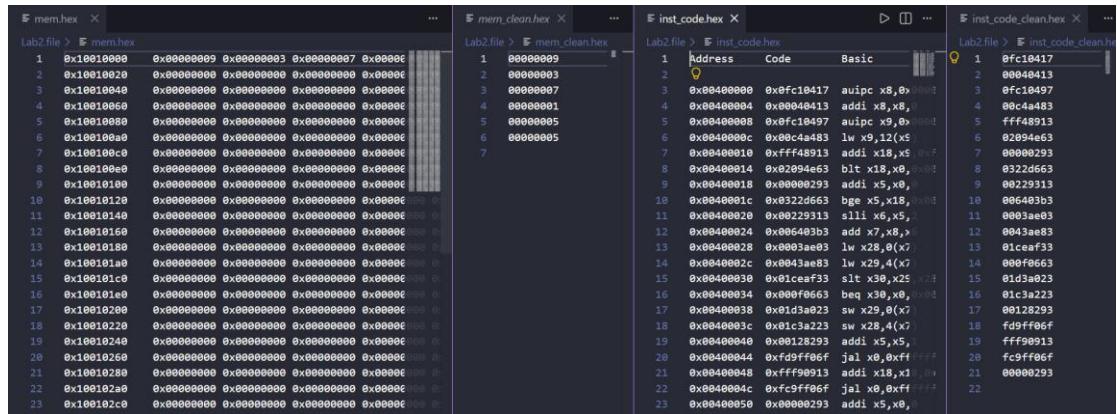


图 5.2-2 指令与数据清洗

5.2.2 仿真结果

使用 Vivado 2019.2 运行仿真。波形图如下所示：

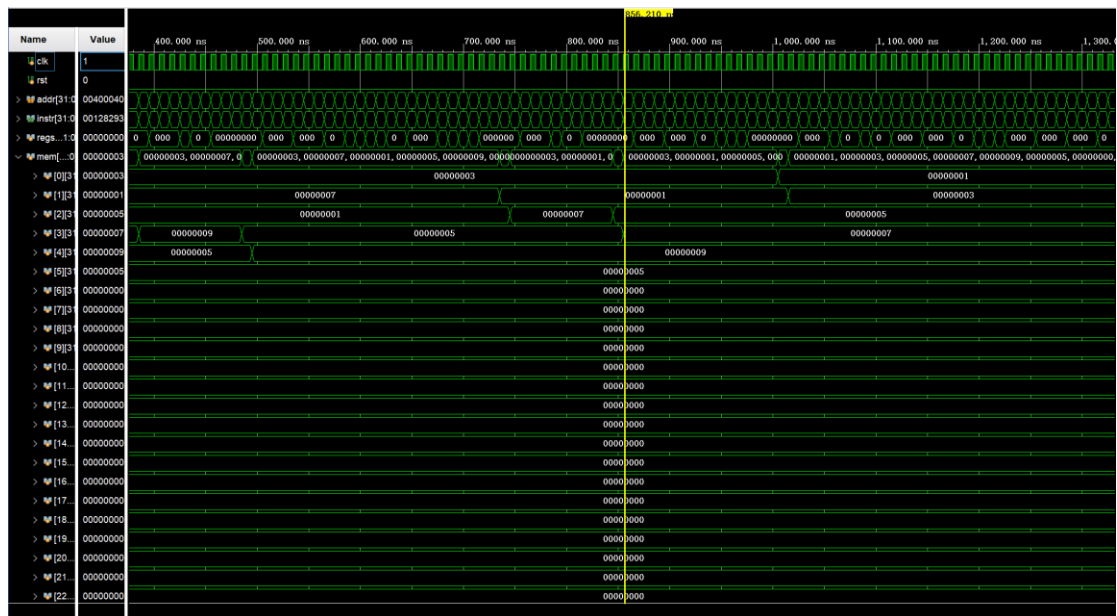


图 5.2-3 系统仿真波形图

从波形观察到：

- CPU 在每一个时钟周期都能够正确获取地址和指令；
- 内层循环中，`lw t3, 0(t2)`和`lw t4, 4(t2)`连续读出相邻两个数组元素，ALU 进行减法比较（通过 `slt` 或 `beq` 后的 `zero` 标志判断是否需要交换）。

- 当需要交换时，sw 指令将两个数写回内存，地址不变。
- 排序结束后，原内存存储的数据被排序为 1、3、5、7、9。

T=955000	PC=00400028	Instr=0003ae03	x5=	0	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	7	x29=	5	x30=	1								
Memory: m1=	3	m2=	1	m3=	5	m4=	7	m5=	9	m6=	5					
T=965000	PC=0040002c	Instr=0043ae83	x5=	0	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	5	x30=	1								
Memory: m1=	3	m2=	1	m3=	5	m4=	7	m5=	9	m6=	5					
T=975000	PC=00400030	Instr=01ceaf33	x5=	0	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	1	x30=	1								
Memory: m1=	3	m2=	1	m3=	5	m4=	7	m5=	9	m6=	5					
T=985000	PC=00400034	Instr=000f0663	x5=	0	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	1	x30=	1								
Memory: m1=	3	m2=	1	m3=	5	m4=	7	m5=	9	m6=	5					
T=995000	PC=00400038	Instr=01d3a023	x5=	0	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	1	x30=	1								
Memory: m1=	3	m2=	1	m3=	5	m4=	7	m5=	9	m6=	5					
T=1005000	PC=0040003c	Instr=01c3a223	x5=	0	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	1	x30=	1								
Memory: m1=	1	m2=	1	m3=	5	m4=	7	m5=	9	m6=	5					
T=1015000	PC=00400040	Instr=00128293	x5=	0	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	1	x30=	1								
Memory: m1=	1	m2=	3	m3=	5	m4=	7	m5=	9	m6=	5					
T=1025000	PC=00400044	Instr=fd9ff06f	x5=	1	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	1	x30=	1								
Memory: m1=	1	m2=	3	m3=	5	m4=	7	m5=	9	m6=	5					
T=1035000	PC=0040001c	Instr=0322d663	x5=	1	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	1	x30=	1								
Memory: m1=	1	m2=	3	m3=	5	m4=	7	m5=	9	m6=	5					
T=1045000	PC=00400020	Instr=00229313	x5=	1	x6=	0	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	1	x30=	1								
Memory: m1=	1	m2=	3	m3=	5	m4=	7	m5=	9	m6=	5					
T=1055000	PC=00400024	Instr=006403b3	x5=	1	x6=	4	x7=	268500992	x8=	268500992	x9=	5	x10=	0	x17=	0
			x28=	3	x29=	1	x30=	1								

图 5.2-4 系统仿真运行结果

经过对比，关键寄存器的值在执行过程中也与 RARS 仿真器运行的结果一致。这进一步证明单周期 CPU 设计正确实现了 RV32I 指令子集（ADD、SUB、ORI、LW、SW、BEQ，以及扩展的 SLT、BLT、BGE 等），并通过了 5 整数排序验证。

第六章 心得体会及总结

本次单周期 CPU 设计实验，我从零开始完成 RISC-V RV32I 指令子集（ADD、SUB、ORI、LW、SW、BEQ，并扩展了 SLT、BLT、BGE、AUIPC、JAL 等）的处理器实现，并成功运行了 5 个整数的冒泡排序程序。

通过此次实验，我对 CPU 的工作原理有了更深入地理解。从最初的指令格式分析，到设计 PC、寄存器堆、ALU、立即数生成器、数据存储器等模块，再到将它们连接成完整的数据通路，我学会了如何将一条指令的二进制编码转换为硬件的控制信号，学习到不同的指令格式（R、I、S、B、U、J）需要不同的立即数提取逻辑。在控制逻辑中，我根据 opcode、funct3、funct7 直接译码产生所有控制信号，让我更清晰地理解了每个控制信号的作用（如 reg_we、mem_we、

alu_src_sel 等)。我也理解了单周期 CPU 控制简单，但由于处理路径长，限制了最高频率。如果要支持更复杂的指令集或更高性能，流水线是必然选择。

综上，这次单周期 CPU 设计实验不仅巩固了课堂所学的计算机组成原理知识，也锻炼了硬件设计和调试能力，为后续学习流水线 CPU 打下了基础。